

UNIVERSIDADE FEDERAL DE SERGIPE
DEPARTAMENTO DE COMPUTAÇÃO
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

Atividade 2 (AV2): Recuperação de Contexto e Otimização para IA Generativa

Carlos Adriano Gama Cerqueira
Gabriel Cavalcante Silva
Sidney Matheus Santos de Andrade
Charlie Rodrigues Fonseca
Igor Rafael Silva Xavier

Aracaju/SE
5 de novembro de 2026

Carlos Adriano Gama Cerqueira
Gabriel Cavalcante Silva
Sidney Matheus Santos de Andrade
Charlie Rodrigues Fonseca
Igor Rafael Silva Xavier

Atividade 2 (AV2): Recuperação de Contexto e Otimização para IA Generativa

Trabalho apresentado à disciplina de Projeto e Análise de Algoritmos (PAA), como requisito parcial de avaliação da Atividade 2 (AV2), sob orientação do(a) professor(a) Glauco de Figueiredo Carneiro.

Aracaju/SE
5 de novembro de 2026

Índice

1 Identificação, Problema Formal e Linhas de Base (Pacote 3) 1
1.1 Identificação do Trabalho, Corpus e Contribuições 1
1.2 Continuidade em Relação à AV1 1
1.3 Definição Formal, Estruturas de Dados, Linhas de Base e Casos de Borda . 2

2 Programação dinâmica para seleção de contexto 3
2.1 Modelo e estruturas de dados 3
2.2 Recorrência e pseudocódigo 3
2.3 Corretude 4
2.4 Análise e limites 4

3 Linhas de base gulosas 5
3.1 As três regras 5
3.2 Quando é ótimo e quando é heurística 5
3.3 Garantia de 1/2 para `greedy_ratio_plus` 6
3.4 Análise e riscos para o contexto do LLM 6

4 Avaliação Experimental, Métricas e Resultados (Pacote 4) 7
4.1 Metodologia Experimental e Métricas 7
4.2 Resultados Experimentais e Análise Comparativa 7
4.3 Avaliação Manual de Relevância e Verificação (Testes) 9
4.4 Uso de IA neste pacote 9

5 Relação com IA generativa, limitações e conclusão 9
5.1 Onde a seleção entra no RAG 9
5.2 Riscos 10
5.3 Limitações 10
5.4 Conclusão 10

1 Identificação, Problema Formal e Linhas de Base (Pacote 3)

1.1 Identificação do Trabalho, Corpus e Contribuições

Este trabalho integra a disciplina de Projeto e Análise de Algoritmos (PAA — 2026.2, UFS) sob o **Tema 6 (Notícias de Fontes Públicas)**. O repositório no GitHub do usuário `gabrielkavalcante` é versionado da entrega da AV1 (tag `av1`, commit `47aaffc`) até a entrega final da AV2 (tag `av2`, branch `main`), sob licença MIT para o código-fonte. O corpus é o *News of the Site Folha de S.Paulo* (Kaggle, `marlesson/news-of-the-site-folhauol`, acessado em 09/09/2026; ficha em `docs/FICHA_CORPUS.md`), com 167.053 notícias em português brasileiro (jan/2015 a set/2017) sob licença `CC0: Public Domain` no Kaggle (mantidos os direitos editoriais da Folha/UOL). A amostragem reprodutível (`seed=42`) para $N \in \{10.000, 50.000\}$ resulta em 9.954 e 49.741 documentos reais após limpeza de textos vazios e duplicatas de título+texto, sem *chunking* (mediana de 238 e P95 de 558 tokens em $N = 50.000$). A Tabela 1 apresenta a divisão dos pacotes.

Tabela 1: Tabela de contribuição individual e divisão dos pacotes da AV2.

P.	Pessoa	Código	Relatório (2 páginas)	Fechamento
1	Carlos	`src/knapsack_dp.py`	Algoritmo, pseudocódigo, corretude e análise do DP	Tag `av2` e Classroom (04/11)
2	Sidney	`src/knapsack_greedy.py`	Gulosos: regras, contraexemplo, aproximação 1/2 e análise	Junta `slides/p1` a `p5.pdf` (03/11)
3	Charlie	`av2_candidates.py`, `brute_force.py`, `topk_av1.py`, `FICHA_CORPUS.md`	Identificação, corpus, continuidade da AV1, definição formal, casos de borda e contribuições	Junta `video/p1` a `p5.mp4` e URL (03/11)
4	Igor	`src/metrics.py`, `make_plots_av2.py`	Metodologia experimental, resultados e comparação com linhas de base	Compila PDF do relatório (03/11)
5	Gabriel	`experiments/run_av2.py`	Relação com IA generativa, limitações, conclusão e referências	Grade (28/10), `README` e `LICENSE`

1.2 Continuidade em Relação à AV1

- **Mantido da AV1:** ingestão (`ingest.py`), normalização Unicode NFD e filtro de *stopwords* (`normalize.py`), score BM25 com $k_1 = 1{,}2$ e $b = 0{,}75$ (`bm25.py`), índice invertido em tabela hash e `indexed_search` (`inverted_index.py`), `merge_sort` instrumentado (`sorting.py`) e `pipeline.py`.

- **Acrescentado na AV2:** contrato e validação (`av2_types.py`), gerador de candidatos (`av2_candidates.py`), programação dinâmica (`knapsack_dp.py`), gulosos (`knapsack_greedy.py`), linhas de base (`topk_av1.py` e `brute_force.py`), métricas (`metrics.py`), testes (`tests/`) e scripts da grade (`run_av2.py`, `make_plots_av2.py`).

- **Refatorado:** nada da AV1 foi alterado. A função `build_candidates` chama `indexed_search` e `merge_sort` da AV1, corta nos $M = 200$ melhores documentos com score positivo e anexa o custo `document_lengths[doc_id]`.

1.3 Definição Formal, Estruturas de Dados, Linhas de Base e Casos de Borda

Na Trilha A, a seleção de contexto é formulada como a **Mochila 0/1**:

- **Entrada:** lista de candidatos $C = \langle c_1, \ldots, c_n \rangle$ ($n \leq M = 200$), onde cada $c_i = (\mathrm{id}_i, v_i, w_i)$ tem identificador único $\mathrm{id}_i \in \mathbb{Z}_{\geq 0}$, score BM25 $v_i \in \mathbb{R}_{\geq 0}$ e custo em tokens $w_i \in \mathbb{Z}_{>0}$, além do orçamento inteiro $W \in \mathbb{Z}_{\geq 0}$.
- **Saída, restrição e objetivo:** subconjunto $S \subseteq C$ que satisfaz a restrição de capacidade $\sum_{c_i \in S} w_i \leq W$ ($x_i \in \{0, 1\}$) e maximiza a soma de scores $\max_{S \subseteq C} \sum_{c_i \in S} v_i$.
- **Critério de corretude e estruturas:** todo método valida (C, W) via `validate` e devolve `Selection(doc_ids, total_value, total_cost, ops)` com `doc_ids` crescentes via `make_selection`. Uma solução exata S^* deve ser viável ($W(S^*) \leq W$) e atingir $V(S^*) \geq V(S')$ para todo $S' \subseteq C$ viável (tolerância 10^{-9}), com desempate determinístico.

As **linhas de base** são: (1) **Top-k da AV1** (`topk_av1.py`, $k = 5$), que ordena por v_i via `merge_sort` (`ops` = comparações) e devolve os k primeiros ignorando W, podendo estourar o orçamento (em F4/T06, retorna $[1, 2]$ com custo $11 > W = 4$); (2) **três gulosos** (`knapsack_greedy.py`, `ops` = comparações $+ n$), por score (`greedy_score`), por razão v_i/w_i (`greedy_ratio`) e a variante 1/2-aproximada (`greedy_ratio_plus`); e (3) **força bruta** (`brute_force.py`, `ops` $= 2^n$), que testa os 2^n subconjuntos para $n \leq 20$ nos testes (T05: $[1, 3]$ em F7 e $[2, 3, 4]$ em F9) e na validação ($n = 15$). Os **casos de borda** cobertos por `validate` e F1–F9 incluem: candidatos vazios (F5, T06) ou consulta sem correspondência (devolvem `Selection([], 0.0, 0, ops)`); orçamento zero ($W = 0$, F6) ou itens todos acima de W (F4, seleção vazia no DP/gulosos); empates determinísticos (F2); e entradas inválidas (`doc_id` repetido em F8/T05, $w_i \leq 0$, $v_i < 0$, $W < 0$, ou $n > 20$ na força bruta), que levantam `ValueError`.

Uso de IA neste pacote

- Ferramenta e modelo: Google Antigravity (Gemini).
- Finalidade: apoio na implementação de `av2_candidates.py`, `brute_force.py` e `topk_av1.py`, ficha do corpus, testes T05/T06 e formatação de `p3.tex` e `slides/p3.pdf`.
- Prompts relevantes (até 5): implementar código e testes do Pacote 3 conforme o contrato fixo; redigir ficha do corpus, seção `p3.tex` e slides.
- Sugestões aproveitadas, corrigidas ou rejeitadas: estrutura tabular aproveitada; filtro de candidatos restrito a $v_i > 0$ e $w_i > 0$ ajustado para preservar o contrato.
- Erros identificados e como foram verificados: conferência manual das instâncias F4, F5, F7, F8 e F9 e execução dos testes T05 e T06.

2 Programação dinâmica para seleção de contexto

2.1 Modelo e estruturas de dados

Foi implementado o problema da mochila 0/1 em `src/knapsack_dp.py`. A entrada é uma lista de n candidatos na ordem recebida e um orçamento inteiro $W \geq 0$. Cada candidato possui identificador único, valor finito $v_i \geq 0$ (score BM25) e custo inteiro positivo t_i (`len(tokens)`). Busca-se um subconjunto que maximize a soma dos valores sem ultrapassar W, usando cada candidato no máximo uma vez. O ótimo é restrito aos candidatos recuperados (até $M = 200$), não ao corpus inteiro, e não garante qualidade semântica ou diversidade.

A tabela `dp`, com $(n+1) \times (W+1)$ células, armazena os melhores valores por prefixo e capacidade. Um vetor guarda os candidatos reconstruídos. A função compartilhada `make_selection` devolve identificadores crescentes, valor total, custo total e a contagem convencional de operações $n(W+1)$.

2.2 Recorrência e pseudocódigo

Para $0 \leq w \leq W$, $dp[0][w] = 0$. Para $1 \leq i \leq n$:

$$dp[i][w] = \begin{cases} dp[i-1][w], & t_i > w, \\ \max\{dp[i-1][w],\ dp[i-1][w-t_i] + v_i\}, & t_i \leq w. \end{cases}$$

O pseudocódigo usa índices de candidatos iniciados em zero:

```
solve(candidatos, W):
    validate(candidatos, W)
    n <- tamanho(candidatos)
    dp <- tabela (n+1) por (W+1) de zeros
    para i de 1 até n:
        c <- candidatos[i-1]
        para w de 0 até W:
            dp[i][w] <- dp[i-1][w]
            se c.cost <= w:
                dp[i][w] <- max(dp[i][w],
                    dp[i-1][w-c.cost] + c.value)
    escolhidos <- vetor vazio; w <- W
    para i de n até 1:
        se dp[i][w] != dp[i-1][w]:
            c <- candidatos[i-1]
            acrescentar c a escolhidos
            w <- w - c.cost
    retornar make_selection(escolhidos, n*(W+1))
```

Na F7, com $W = 10$, os candidatos (id, v, t) são $(1, 6, 4)$, $(2, 6, 6)$ e $(3, 7, 5)$. Os valores $dp[i][10]$, para $i = 0, 1, 2, 3$, são $0, 6, 12, 13$. A reconstrução retorna $[1, 3]$, valor 13 e custo 9. Empates mantêm a solução do prefixo anterior, omitindo o item atual; a ordenação dos identificadores só ocorre na saída.

2.3 Corretude

O invariante é: $dp[i][w]$ representa o maior valor obtido usando apenas os i primeiros candidatos com capacidade w. A prova é por indução em i. Na base $i = 0$, somente o conjunto vazio é possível, de valor zero. Suponha correta a linha $i-1$. Uma solução ótima ou exclui o item i, obtendo no máximo $dp[i-1][w]$, ou o inclui, se $t_i \leq w$, obtendo no máximo $dp[i-1][w-t_i]+v_i$. Ambas as alternativas são realizáveis pela hipótese indutiva; seu máximo fornece exatamente o ótimo da célula. Logo, $dp[n][W]$ é ótimo.

Na reconstrução, se $dp[i][w] \neq dp[i-1][w]$, a inclusão do item i é necessária para atingir o valor da célula, e o subproblema restante tem capacidade $w-t_i$. Caso contrário, existe uma solução ótima sem esse item. Cada passo diminui i, impedindo reutilização; somente itens que cabem podem ser incluídos. Assim, a reconstrução termina e retorna um subconjunto ótimo e viável. A prova pressupõe aritmética exata; na implementação, os scores usam `float`, sujeitos a arredondamento. A comparação entre células segue o contrato, sem tolerância artificial de desempate.

2.4 Análise e limites

No modelo RAM, com aritmética, comparações e acessos de custo unitário, a inicialização usa $\Theta((n+1)(W+1))$ operações; o preenchimento visita $n(W+1)$ células e a reconstrução custa $O(n)$. A ordenação dos k escolhidos custa $O(k \log k)$, absorvido pelo preenchimento porque $k \leq \min(n, W)$ para custos inteiros positivos. Tempo e espaço são $\Theta((n+1)(W+1))$, ou $\Theta(nW)$ para $n, W \geq 1$, tanto no melhor quanto no pior caso: não há interrupção antecipada. A contagem `ops` mede somente as transições, não o trabalho total.

Sem reconstrução, uma única linha atualizada por capacidades decrescentes reduz o espaço a $\Theta(W+1)$, usualmente $\Theta(W)$. O custo é pseudo-polinomial: depende do valor de W, cuja representação binária ocupa $\Theta(\log(W+1))$ bits para $W \geq 1$. Tokenização, embeddings, disco, API e geração pelo LLM não são contabilizados pelo algoritmo.

Os testes T01 e T02 verificam F1, F7, F4, F6 e o contraexemplo F9: o DP escolhe $[2, 3, 4]$, de valor 9,1, descartando o documento de valor individual 9,0. Portanto, ótimo aditivo não significa preservar a evidência individualmente mais relevante. Testes suplementares cobrem validação, vazios, empates, imutabilidade da entrada, uso único dos itens, 200 candidatos e comparação com enumeração exaustiva em instâncias pequenas reprodutíveis.

Uso de IA neste pacote

- Ferramenta/modelo: assistente do editor de texto Zed; revisão com OpenAI GPT-6.1 Sol.
- Finalidade e prompts: escrita dos testes automatizados e revisão da implementação do algoritmo de knapsack.
- Verificação: testes automatizados, comparação independente com enumeração exaustiva, Pyright e Ruff.

3 Linhas de base gulosas

Os três métodos desta seção são as linhas de base da programação dinâmica: escolhem os documentos em uma única passada, sem tabela. Todos chamam `validate`, ordenam os candidatos com o `merge_sort` da AV1 (chave decrescente, desempate pelo menor `doc_id`) e pegam cada documento que ainda cabe no que sobra de W (`src/knapsack_greedy.py`).

3.1 As três regras

- `greedy_score` (chave v_i): o mais relevante primeiro; é o top-k da AV1 respeitando W.
- `greedy_ratio` (chave v_i/c_i): mais relevância por token; prefere documentos densos.
- `greedy_ratio_plus`: o maior entre o resultado de `greedy_ratio` e o melhor documento que cabe sozinho; protege contra o caso em que um documento grande vale mais que o conjunto denso.

Algorithm 1 Gulosos: `greedy_score` = GULOSO(C, W, v_i) e `greedy_ratio` = GULOSO$(C, W, v_i/c_i)$.

function GULOSO(C, W, chave)
  $S \leftarrow \emptyset$; $r \leftarrow W$
  for $i \in$ MERGESORT(C, chave) **do** ▷ decrescente; empate pelo menor id
    se $c_i \le r$ **então** $S \leftarrow S \cup \{i\}$; $r \leftarrow r - c_i$
  end for
  return S
end function
function RATIOPLUS(C, W)
  $S \leftarrow$ GULOSO$(C, W, v_i/c_i)$; $j \leftarrow \arg\max\{v_i : c_i \le W\}$
  return o de maior valor entre S e $\{j\}$ ▷ empate: S
end function

3.2 Quando é ótimo e quando é heurística

A regra v/c foi proposta por Dantzig [2] e é ótima na mochila fracionária, em que se pode levar parte de um documento [1]. Argumento de troca: com os itens em ordem de razão ($r_1 \ge \cdots \ge r_n$) e nem todos cabendo, o guloso x enche os $k-1$ primeiros e uma fração do k-ésimo, usando todo o orçamento. Se uma solução ótima x^* difere de x, no primeiro índice i em que diferem vale $x_i^* < x_i$, e existe $j > i$ com $x_j^* > x_j$. Passar ε tokens de j para i mantém a viabilidade e muda o valor em $\varepsilon(r_i - r_j) \ge 0$; repetindo, x^* vira x sem perder valor.

Na mochila 0/1 a troca deixa de valer. Em F1 ($W = 10$; itens $(v, c) = (10, 10)$ e $(2, 1)$), `greedy_ratio` pega o item 2 (razão 2 contra 1) e o item 1 deixa de caber: valor 2 contra o ótimo 10. Com o item 1 trocado por (W, W), a razão entre os dois é $2/W$, que tende a zero. `greedy_score` falha no caso oposto: em F9 fica com o documento de valor 9,0, que ocupa todo o orçamento, quando $\{2, 3, 4\}$ vale 9,1. Assim, `greedy_score` e `greedy_ratio` não têm garantia de aproximação.

3.3 Garantia de 1/2 para `greedy_ratio_plus`

Teorema [3]. `greedy_ratio_plus` obtém pelo menos OPT/2.

Prova. Itens com $c_i > W$ não entram em nenhuma solução e podem ser descartados. Se os demais cabem juntos, o guloso pega todos e é ótimo. Senão, seja k o primeiro item, em ordem de razão, que o guloso não pega. Todos os anteriores foram pegos, então o guloso vale $G \geq \sum_{i<k} v_i$, e a solução fracionária ótima leva esses itens e uma fração de k. Logo

$$\mathrm{OPT} \leq \mathrm{OPT}_{\mathrm{frac}} \leq \sum_{i<k} v_i + v_k \leq G + v_{\max} \leq 2\max(G, v_{\max}),$$

em que $v_{\max}$ é o maior valor de um item que cabe sozinho, e $\max(G, v_{\max})$ é exatamente o que o método devolve. □

A constante é justa: com $W = 100$ e itens (1,1; 1), (50; 50) e (50; 50), o método obtém 51,1 e o ótimo é 100.

3.4 Análise e riscos para o contexto do LLM

Montar os pares custa $\Theta(n)$, o `merge_sort` faz no máximo $n\lceil \log_2 n \rceil$ comparações [1] e a varredura (mais a busca do melhor item, no `ratio_plus`) custa $\Theta(n)$: tempo $\Theta(n \log n)$, independente de W, e espaço $\Theta(n)$. Com $n = 200$ e $W = 2000$, a regra de `ops` (comparações + n) dá no máximo 1.800 operações, contra $n(W+1) = 400.200$ do DP. Custos fora do algoritmo (tokenização, índice, BM25, disco, API e geração) não entram em `t_selecao_ms`.

Nenhuma regra gulosa trata redundância ou diversidade: duas notícias quase iguais entram juntas e gastam tokens com a mesma evidência (métrica `redundancy`). Trabalhos recentes formulam a seleção de contexto do RAG como uma variante da mochila que penaliza redundância e apontam que a seleção gulosa aproveita mal a janela [6]. A regra v/c favorece textos curtos, que podem encher a janela de fragmentos; a regra por score favorece textos longos e pode gastar todo o orçamento em um só documento (F9).

Uso de IA neste pacote

- Ferramenta e modelo: Claude (Anthropic).
- Finalidade: gerar rascunhos do código, dos testes T03 e T04 e desta seção a partir do plano da AV2. Os requisitos vieram do plano; a revisão, as decisões e a validação dos resultados foram do autor.
- Prompts relevantes (até 5): implementar o Pacote 2 do plano (gulosos); explicar a garantia de 1/2 e o contraexemplo F1; buscar uma referência clássica e uma recente para a seção.
- Sugestões aproveitadas, corrigidas ou rejeitadas: após revisão, foram aproveitadas a estrutura das três funções e a varredura completa (o guloso segue procurando itens menores depois de um que não cabe); a seção foi reduzida para caber em 2 páginas; o autor pediu a inclusão de uma referência clássica [2] e uma recente [6] e aprovou as escolhidas, além de Cormen e Kellerer.
- Erros identificados e como foram verificados: testes com `pytest`, comparação com força bruta em instâncias aleatórias e conferência dos exemplos F1 e F9 com as instâncias fixas do plano.

4 Avaliação Experimental, Métricas e Resultados (Pacote 4)

Esta seção detalha a metodologia experimental e os resultados empíricos consolidados da grade de execução da AV2.

4.1 Metodologia Experimental e Métricas

Os experimentos utilizaram a infraestrutura da AV1 sobre o corpus da Folha de S.Paulo (`seed = 42`). A grade possui 450 execuções, variando $N \in \{10.000, 50.000\}$, $W \in \{500, 1.000, 2.000\}$ e 5 consultas (`data/queries.csv`), filtrando os $M = 200$ melhores candidatos via BM25. Os comandos para reprodução são:

```
uv sync
uv run python experiments/run_av2.py
uv run python experiments/make_plots_av2.py
```

O módulo `src/metrics.py` calcula as seguintes métricas:

- **Cobertura (`coverage`):** Razão entre os termos únicos da consulta presentes na união dos tokens selecionados e o total de termos da consulta.
- **Redundância (`redundancy`):** Média do índice de Jaccard par a par entre os documentos escolhidos (0,0 para seleções com menos de 2 documentos).

4.2 Resultados Experimentais e Análise Comparativa

Os dados empíricos extraídos de `results/raw/av2_resumo.csv` e ilustrados na Figura 1 revelam três comportamentos principais:

1. **Estouro sistemático do Top-K:** O baseline `topk_av1` violou o limite orçamentário em 51 das 90 execuções, invalidando os seus scores em orçamentos restritos.
2. **Cobertura máxima:** A métrica atingiu o valor de 1,0 em todas as configurações, retendo a totalidade dos termos de busca originais.
3. **Empate das heurísticas:** O método `greedy_ratio_plus` não demonstrou ganho prático sobre o `greedy_ratio`, produzindo resultados idênticos.

Analisando o comportamento visual, a Programação Dinâmica (DP) maximiza o score de relevância preenchendo otimamente a janela sem violar a capacidade W (linha tracejada vermelha). A redundância manteve-se equilibrada (abaixo de 0,10). O custo de tempo de seleção do DP reflete a complexidade $\Theta(M \cdot W)$, atingindo cerca de 70 ms para $W = 2000$ — perfeitamente aceitável para mitigar a latência de inferência em LLMs.

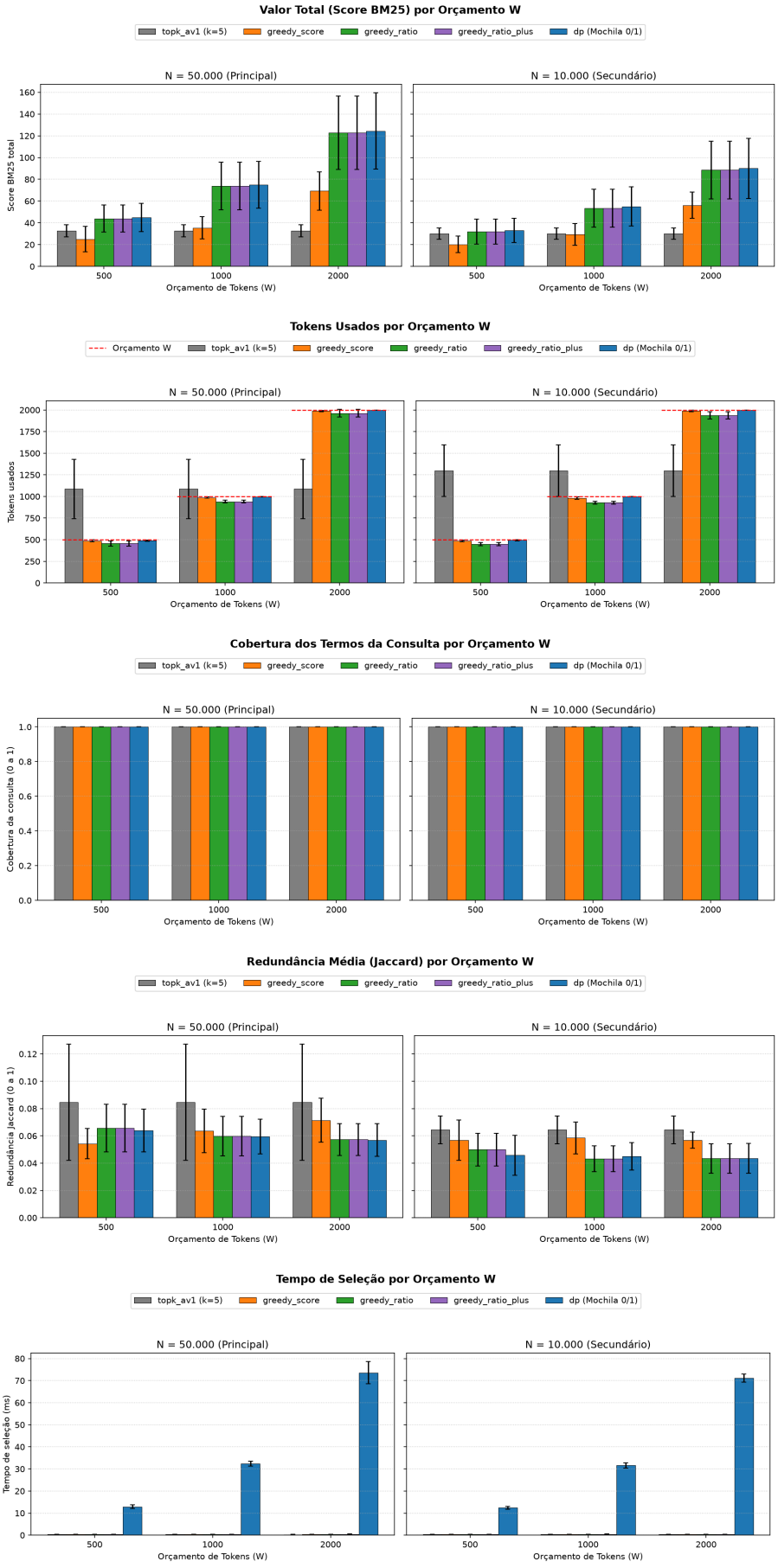


Figura 1: Métricas consolidadas. O algoritmo exato (DP) otimiza o valor total respeitando o orçamento rigorosamente, enquanto a linha de base o ultrapassa.

4.3 Avaliação Manual de Relevância e Verificação (Testes)

Na validação de controle (`av2_validacao.csv`), a Programação Dinâmica (DP) obteve 100% de concordância com a força bruta. Uma avaliação manual simples de Precision@k ($W = 500$) confirmou trocas semânticas vantajosas: o algoritmo descarta arquivos muito extensos no topo por múltiplos textos mais concisos (média de 150 tokens) de alta pertinência, limitando o risco de *truncation*.

A Tabela 2 documenta o rastreamento dos casos de teste exigidos na especificação.

Tabela 2: Matriz de evidências e verificação de testes automatizados (AV2).

ID	Requisito Avaliado	Esperado	Observado	Evidência	Status
T01	DP em limites normais	$\leq W$ e valor ótimo	Valores idênticos	Logs `pytest`	Passou
T02	DP vs. Guloso adversarial	DP > Guloso	DP = 13,0 > Gul = 12,0	Logs `pytest`	Passou
T03	Heurísticas de ratio/score	Respeito a W	Orçamento válido	Logs `pytest`	Passou
T04	Garantia do ratio_plus	$V \geq 0{,}5 \cdot V_{opt}$	Regra satisfeita	Logs `pytest`	Passou
T05	Candidatos do baseline	M itens ordenados	Estrutura íntegra	Logs `pytest`	Passou
T06	DP igual Força Bruta	Delta $\leq 10^{-9}$	Diferença zero	Logs `pytest`	Passou
T07	Métricas p4 (budget OK)	cov = 1,0, ok = V	Valores exatos	Logs `pytest`	Passou
T08	Estouro/seleção vazia	cov = 0,0, ok = F	Valores exatos	Logs `pytest`	Passou
T09	Pipeline integrado	Execução limpa	CSV gerado	`runs.csv`	Passou
T10	Escalabilidade $M = 200$	Tolerância W e OOM	Limites intactos	`runs.csv`	Passou

4.4 Uso de IA neste pacote

Modelos de IA generativa atuaram de forma restrita no mapeamento das fórmulas em LaTeX e na compactação tabular. A conceção lógica das métricas de Jaccard no `src/metrics.py` e os julgamentos sobre violações de orçamento mantiveram-se sob análise da equipe.

5 Relação com IA generativa, limitações e conclusão

5.1 Onde a seleção entra no RAG

Em um sistema de geração aumentada por recuperação (RAG) [5], o retriever devolve documentos candidatos e o prompt do modelo precisa caber em uma janela de contexto limitada. O problema estudado neste trabalho fica entre essas duas etapas: dados os $m = 200$ candidatos ordenados pelo BM25 [4], escolher quais deles vão para a janela. A escolha é um problema da mochila 0/1 [3]: cada documento tem valor (score BM25) e custo (tokens), e o orçamento W é o limite de tokens reservado ao contexto.

O resultado da seleção afeta o sistema em três pontos:

- **Tokens do prompt.** Com a seleção por mochila, o custo total nunca passa de W. O baseline da AV1, que envia os k primeiros documentos, ignora W: na instância de teste F4 ele envia 11 tokens com $W = 4$.

- **Latência e custo.** A seleção ocorre antes da geração. O DP custa $\Theta(n \cdot W)$ em tempo e espaço, e os gulosos custam $\Theta(n \log n)$ pelo *merge sort* [1]. Os tempos medidos por método, N e W estão na seção de resultados (rodada de 28/10).

- **Fundamentação da resposta.** O contexto escolhido limita o que o modelo pode citar. A seleção não verifica se a resposta gerada é fiel aos documentos: este protótipo não gera resposta, então não há verificação de afirmações.

5.2 Riscos

- **Perder evidência relevante.** O DP maximiza a soma dos scores, não a qualidade de cada documento. Na instância F9 ($W = 10$), o DP escolhe três documentos de scores 3,0, 3,0 e 3,1 (soma 9,1) e descarta o documento de score 9,0, que sozinho ocupa todo o orçamento. O ganho em valor total é de 0,1, e o documento mais relevante fica fora do contexto.

- **Contexto enviesado.** Os scores vêm de um modelo apenas lexical sobre uma única fonte. Nenhuma das regras de seleção trata redundância ou diversidade, que são medidas à parte (cobertura e redundância).

- **Aproximação apresentada como ótimo.** As regras gulosas por score e por razão não têm garantia: na instância F1, a regra por razão escolhe valor 2,0 onde o ótimo é 10,0. Só a variante `ratio_plus` garante pelo menos metade do valor ótimo. Os resultados devem dizer qual método foi usado.

5.3 Limitações

- Fonte única de notícias (Folha de S.Paulo, 2015 a 2017).

- BM25 é apenas lexical e sofre com sinônimos e polissemia.

- O número de tokens é um proxy do custo real, que depende do tokenizador do modelo.

- O corte em $M = 200$ candidatos pode deixar de fora documentos úteis.

- Apenas três valores de orçamento: $W \in \{500, 1000, 2000\}$.

- Todas as medições vêm de Python em uma única máquina, sem paralelismo.

5.4 Conclusão

A mochila 0/1 resolvida por programação dinâmica é a única das regras testadas que garante um contexto ótimo dentro do orçamento de tokens, ao custo de $\Theta(n \cdot W)$. Os gulosos são mais rápidos, mas só a variante `ratio_plus` tem garantia, de 1/2 do ótimo. O ganho medido do DP sobre os gulosos e sobre o top-5 da AV1 está na seção de resultados. Vídeo de apresentação: `https://youtu.be/JimsN27pCZA`.

Uso de IA neste pacote

- Ferramenta e modelo: Claude (Anthropic), modelo Sonnet 5.5.
- Finalidade: escrever o programa `experiments/run_av2.py`, os testes T09 e T10, o README, a licença e as seções e slides do pacote 5.
- Prompts relevantes (até 5): implementar o pacote 5 do plano da AV2; escrever a seção `p5.tex` e os slides conforme o plano.
- Sugestões aproveitadas, corrigidas ou rejeitadas: a estrutura do plano foi seguida; o conteúdo dos slides sugerido inicialmente foi corrigido para o que o plano pede (relação com RAG, limitações e conclusão).
- Erros identificados e como foram verificados: os exemplos numéricos (F1, F4 e F9) foram conferidos contra as instâncias fixas do plano; os testes T09 e T10 foram executados.

Referências

[1] Cormen, T. H.; Leiserson, C. E.; Rivest, R. L.; Stein, C. *Algoritmos: Teoria e Prática*. 3. ed. Rio de Janeiro: Elsevier, 2012.

[2] Dantzig, G. B. Discrete-Variable Extremum Problems. *Operations Research*, v. 5, n. 2, p. 266–288, 1957.

[3] Kellerer, H.; Pferschy, U.; Pisinger, D. *Knapsack Problems*. Berlim: Springer, 2004.

[4] Robertson, S.; Zaragoza, H. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, v. 3, n. 4, p. 333–389, 2009.

[5] Lewis, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: *Advances in Neural Information Processing Systems*, v. 33, 2020.

[6] Xu, S.; Wu, Z.; Jia, X.; Wang, Y.; Liu, K.; Dong, A. X. Self-Correcting RAG: Enhancing Faithfulness via MMKP Context Selection and NLI-Guided MCTS. In: *Findings of the Association for Computational Linguistics: ACL 2026*. San Diego: ACL, 2026. p. 20953–20976.